Claude Code in VS Code

The tricks I use every day

A practical tour for friends: start, configure, orchestrate, amplify. Code first, with non-coding examples too

Four blocks: start, configure, orchestrate, then amplify

1

Start

Claude's tools, extension vs CLI, opening a project

Slides 3-7

2

Configure

.claude/ folder, CLAUDE.md, skills, agents, hooks

Slides 8-14

3

Orchestrate

The Ralph loop: docs, initialize, plans, progress.md, verify

Slides 15-25

4

Amplify

Context, flags, MCP (Drive, Gmail), review, remote

Slides 26-32

Code first, but every block also works beyond code: reports, research, spreadsheets, Word files, email, data.

1

Start

Tools, VS Code vs CLI, first run

In this section

What Claude can do

Extension vs CLI

Open a folder

Start the CLI

Claude works through tools: read, search, edit, run

Read

Opens files: code, images, PDFs, Word, Excel

Grep

Searches inside files for text or patterns

Glob

Finds files by name pattern

Edit

Changes an exact piece of an existing file

Write

Creates or fully replaces a file: code, report, doc

Bash

Runs terminal commands: tests, git, uv, scripts

AskUserQuestion

Asks you to choose between options

Agent

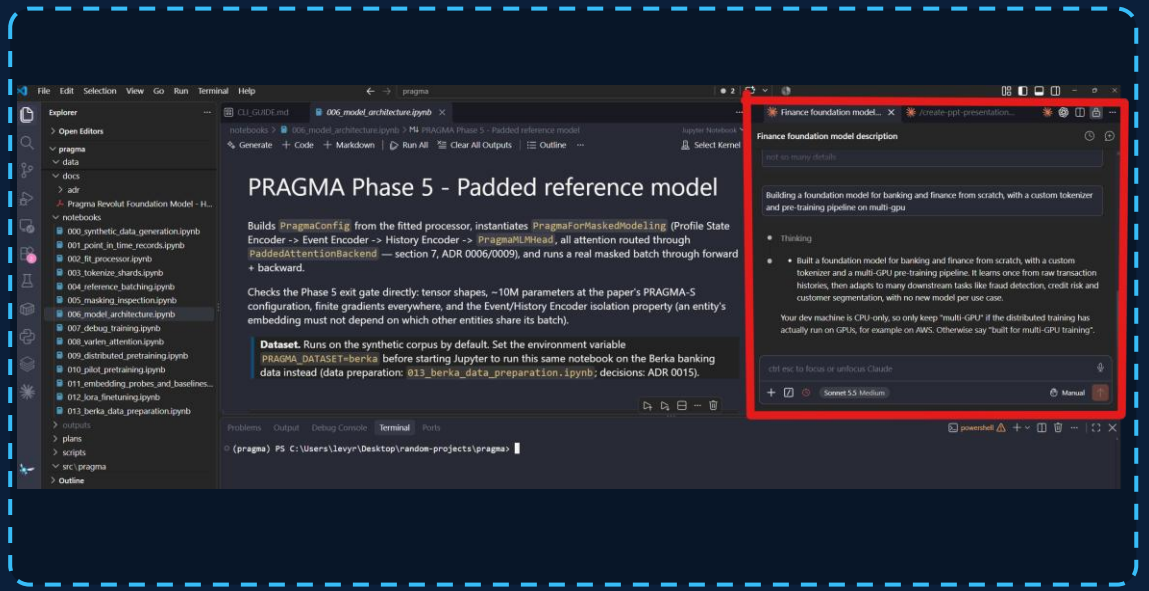
Hands a task to a sub-agent

Also: WebSearch, WebFetch, Skill... You decide which tools may run without asking (permissions).

Two front doors, same Claude: extension and CLI

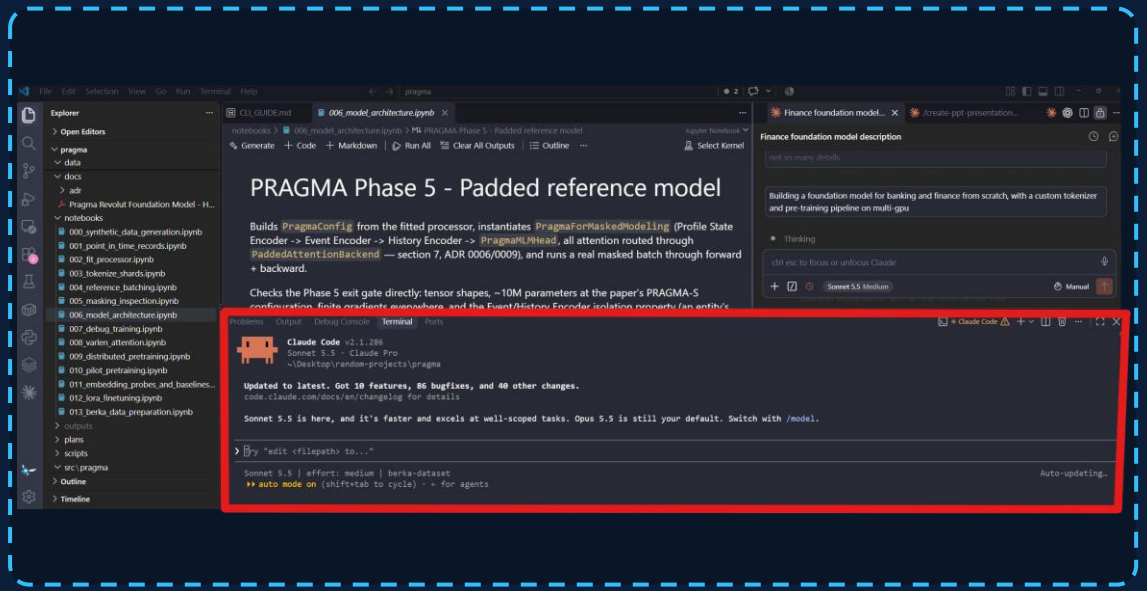
VS Code extension

Chat panel on the right side, inline diffs, @-mention files



CLI in a terminal

Run `claude` in any terminal, including the one inside VS Code



My split: extension for chat day-to-day, CLI for flags, agents and automation

Start in VS Code: open a folder, open the Claude panel

Step 1

File > Open Folder...

Any folder works: code, reports, research notes.

Step 2

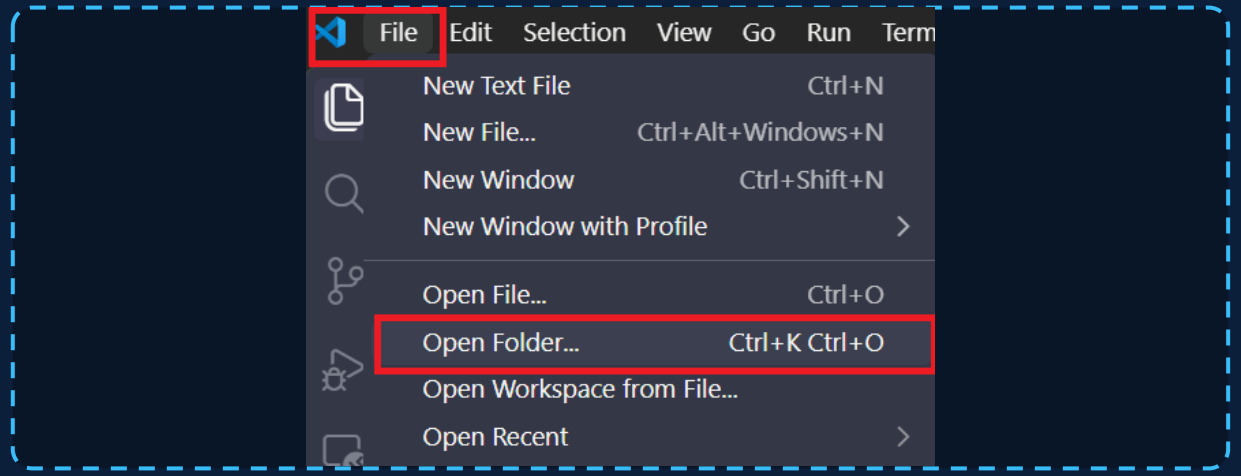
Open the Claude panel

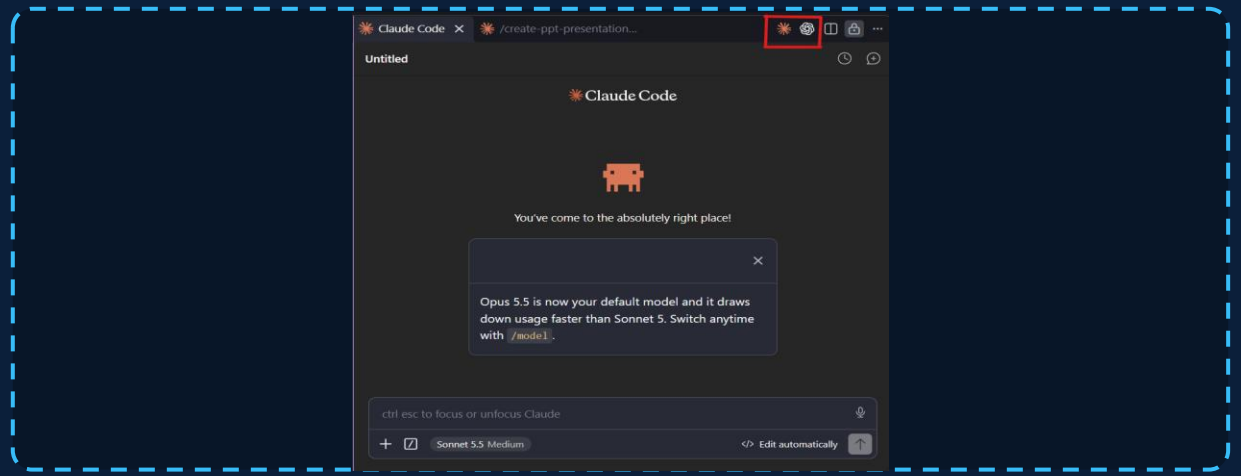
Claude icon in the side menu (right side).

Step 3

Describe a task

Review diffs, then accept or push back.

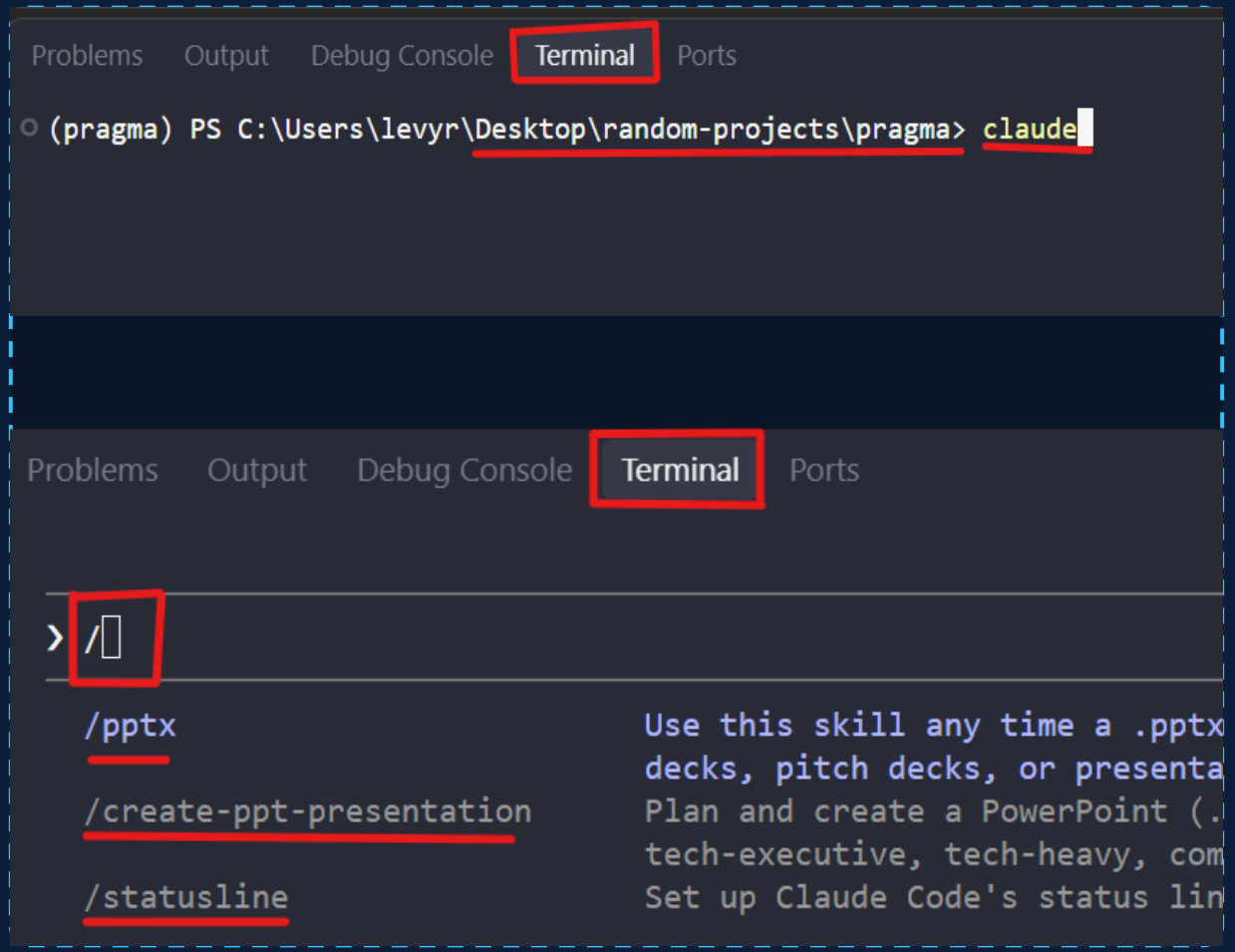




Start the CLI: one word in any terminal

Three moves

- Open a terminal (VS Code: Ctrl+`)
  - Or go to ‘Terminal’→ ‘New Terminal’
- Open folder
  - Or cd into your project folder
- Type claude and press Enter
- Type / to see every slash command



2

Configure

Shape how Claude behaves in your project

In this section

- .claude/ folder
- CLAUDE.md
- Skills, rules, hooks
- My skills
- Agents
- Hooks and settings

The .claude/ folder is where you shape Claude's behavior

```
.claude/
  settings.json
  settings.local.json
  agents/
    summarizer.md
  skills/
    my-skill/SKILL.md
  rules/
    style.md
  commands/
CLAUDE.md  (project root)
```

`settings.json`	Permissions, hooks, env. Shared via git
`settings.local.json`	Your personal overrides, not committed
`agents/`	Sub-agents, one markdown file each
`skills/`	Repeatable playbooks, one folder each
`rules/`	Small focused instruction files
`commands/`	Your own slash commands

Same layout in ~/.claude/ applies to all your projects

CLAUDE.md: Claude's briefing note for your project

What goes in it

- What the project is and its layout
- Commands: how to run, test, lint
- Conventions and style preferences
- Where the plans and decisions live
- Do's and don'ts (e.g. never commit unasked)
- Non-code work: audience, tone, templates, terms

Why it's worth having

- Loaded every session: no re-explaining
- Shared with the team through git
- Fewer wrong guesses, fewer corrections
- A global one in ~/.claude/ holds personal habits

```
> Create a CLAUDE.md for this project: what it is, how to run and test it,
  and the conventions you see in the code. (or just type /init)
```

Skills, rules, hooks, agents: pick the right block

Repeatable behavior

Skills

A playbook Claude loads when the task matches, or when you type /name.

Isolated behavior

Rules

Small focused instruction files, optionally scoped to certain files.

Mandatory behavior

Hooks

Commands that run automatically on events. Guaranteed, not suggested.

Delegated work

Agents

Specialist helpers with their own context, tools and model.

CLAUDE.md = always-on context. These four = on-demand or automatic behavior.

Example of my skills: Claude wrote them, and I keep asking it to edit them

Skill	What it does
initialize-project	After the talk: README, diagrams, decisions, plans/, progress.md and folders
create-notebook	Builds local or Kaggle Jupyter notebooks
create-tests / run-tests	Writes tests; runs pytest + ruff + mypy like CI
create-cli	Adds a small CLI plus a CLI.md on how to run it
create-ppt-presentation	Plans and builds a .pptx deck (like this one)
create-video	Narrated explainer video from a script (MP4)
git-flows	Checks GitHub identity, branch and commit conventions
Non-code ideas	weekly-report, meeting-notes, email-in-my-voice: raw notes in, clean document out

Ask: "Create a skill that does X every time I ask for Y". Later: "Edit that skill so it also Z".

Agents: a markdown file in .claude/agents/

```
---
name: summarizer
description: Summarize files in 5
bullets
tools: Read, Grep, Glob
model: haiku
---
You read the given files and
return a 5-bullet summary.
```

Ask Claude to write it, or use /agents

Built-in agents

- Explore: fast read-only search
- Plan: designs an approach
- General-purpose: multi-step work

How to call them

- @agent-summarizer in the chat
- claude --agent coordinator for the whole session
- Or just ask: Claude delegates by itself

Hooks and settings: get a push notification when Claude finishes

```
{
  "hooks": {
    "Stop": [{
      "hooks": [{
        "type": "command",
        "command": "<your notify script>"
      }]
    }]
  }
}
```

Events: Stop, Notification, PreToolUse...

Settings levels

- ~/.claude/settings.json: all projects
- .claude/settings.json: this project, shared
- .claude/settings.local.json: just you

Easiest way

- Ask: "Add a hook that notifies me when you finish"
- Use /hooks to see what's active
- Handy for any long job: a report, a research run

3

Orchestrate

The **Ralph loop**: plan, progress.md, verify, repeat

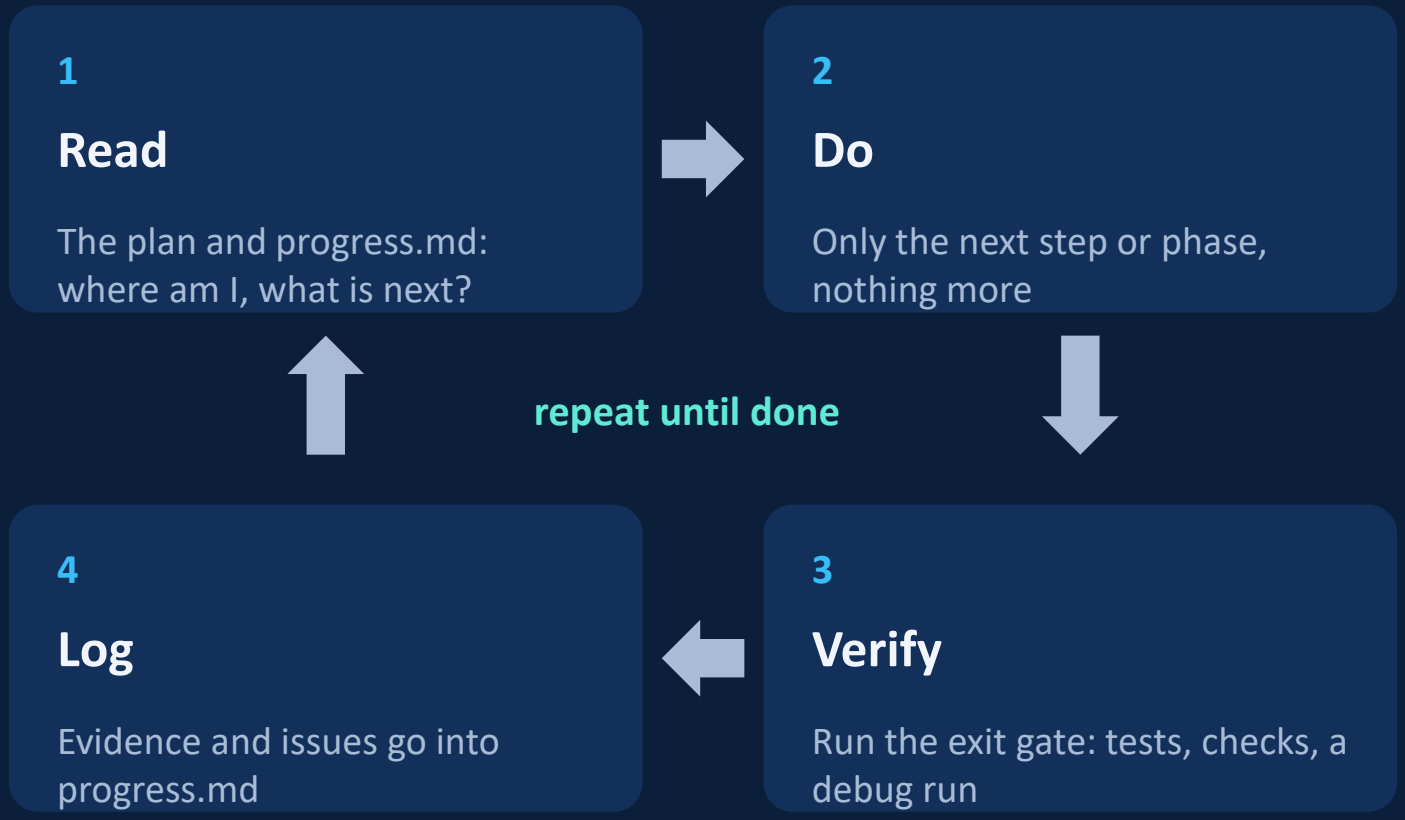
The Ralph loop, my way

1. Add documents to docs/ and data/
2. Talk the problem: ideas, doubts, decisions
3. Say "initialize": docs, folders, plans
4. Detailed plan in plans/, numbered
5. progress.md tracks everything
6. Iterate: build, verify, self-correct
7. Next phase, or many phases at once

The Ralph loop: keep looping until the gates pass

What it is

- Run the AI in a loop until the job is really done
- Each pass starts clean: the memory lives in files, not in the chat
- Mine adds three things: a written plan, exit gates, progress.md
- Works for reports, research and data, not only code



There is a /ralph-loop plugin that automates it. I run the same idea by hand, phase by phase.

CLAUDE.md is the conductor: it makes the loop automatic

```
# my CLAUDE.md (excerpt)

## Planning big work
- plans/001-plan-<name>.md, numbered
- Never rewrite an old plan
- plans/progress.md for long work
- Phases + test plan + exit gates
- Keep progress.md current: status,
  evidence, decisions, known issues

## Working style
- Test on a small sample first
- Ask only when blocked
- No commit unless I ask
- Push notify when done or blocked
```

Plan goes in plans/, numbered

→ Every big job starts as a written plan

Phases, test plan, exit gates

→ Every phase has a pass-or-fail check

Keep progress.md current

→ Evidence is logged after every phase

Small sample, report honestly

→ Failures are shown and fixed, not hidden

Ask only when blocked, notify

→ Long runs go on without me, pinging when needed

Loaded every session, so "next phase" is enough. Global ~/.claude/CLAUDE.md = my habits; a project one wins on conflict.

Start every job the same way: material in, doubts out

1 Put the material in folders

- **docs/** briefs, specs, templates, PDFs, Word files, examples
- **data/** CSVs, spreadsheets, exports, samples
- Claude reads them: nothing to re-explain

2 Talk it through

- Describe the problem and who it is for
- Brainstorm ideas and options together
- Claude asks me questions until the doubts are solved
- Decisions are made out loud; the next step writes them down

```
> Read docs/ and data/. The problem: my monthly client report takes two days.
  Don't plan or build yet: ask me what is unclear and give me options.
```

Code or not: a client brief and a spreadsheet are as good a start as a repository.

Then say “initialize”: docs and project, done

Input: talk, files, or both

- **Talked it through** ideas, doubts and decisions are in the chat
- **Or no talk at all** prints, briefs and notes pasted into docs/ and data/
- Either is enough: I just ask Claude to initialize

Output: docs, folders, plans

- **Docs** README, architecture, flow diagrams, decisions, open questions
- **Folders** docs/ data/ src/ notebooks/ outputs/ test/ plans/
- **Plans** 001-plan-<name>.md and progress.md
- Claude writes whatever the project needs

```
> Initialize the project. Use our conversation and what is in docs/ and data/.
  Write the README, diagrams, decisions and open questions, then the folders and plans.
```

The skill is "initialize-project": it runs when I ask, so every project starts the same way.

Step 4: a detailed plan, numbered and named

```
plans/
  001-plan-report.md
  002-plan-dashboard.md
  progress.md
```

My instructions for plans

- One file per effort: 001, 002, 003...
- Never rewrite an old plan for new work
- Phases, each with tests and an exit gate
- Big work also gets progress.md

```
# 001-plan-monthly-report

## Phase 1: load and clean data
- Write load.py, clean the sales sheet
- Tests: row count = source
- Exit gate: pytest, ruff, mypy pass

## Phase 2: build the Word report
- Fill the template from docs/
- Exit gate: all 6 fields filled,
  numbers match the sheet

## Phase 3: review and export
- Exit gate: no TODO text left
```

Ask Claude: "Make a detailed plan in phases, with a test plan and an exit gate for each."

progress.md: the file that drives the whole loop

```
# progress.md
Status: phase 2 of 4, in progress

## Done
- P1 data loaded and cleaned

## In progress
- P2 build the Word report

## Next
- P3 review, P4 export

## Gate results (evidence)
- P1: pytest 12 passed, ruff clean

## Decisions
- Use the Word template in docs/

## Known issues
- 3 rows miss Client ID: fix in P2
```

Memory outside the chat

A new chat or a new day: Claude re-reads it and continues

Evidence, not promises

Every "done" comes with its gate result next to it

Readable in 30 seconds

I check status and evidence, then say what comes next

Fuel for self-correction

Known issues and failed gates become the next pass's to-do list

Each pass: build, verify, self-correct, repeat

1 Read

plans/ and progress.md: where am I, what is next?

2 Do

Only the next step or phase, nothing more

3 Verify

Run the exit gate: tests, checks, a debug run

Gate fails

Self-correct

Read the error, fix it, log it under Known issues, run the gate again

Gate passes

Log and move on

Write the evidence in progress.md, mark the phase done, next phase

Self-healing: Claude finds and fixes its own mistakes inside the loop, before I review.

Exit gates: a short pass-or-fail check for every phase

Code

- pytest passes
- ruff and mypy are clean
- Debug-mode run works on a small sample
- The CLI command runs end to end

Documents, data, anything else

- Word doc: field "Client" and date are filled in
- Every slide has a title and speaker notes
- Spreadsheet totals match the source file
- No TODO or placeholder text left
- Email: the ask is in the first line

Catch early

Mistakes show up in the loop, not at the end

Self-correct

A failed gate means fix and re-run

Proof

Results sit in progress.md for me to read

Tip: ask Claude to write the check itself: "write a script that checks the report against the brief".

One phase at a time, or all phases at once

Careful: one phase, then "next phase"

- I read the evidence after every phase
- Best for new, risky or paid-API work
- Easy to steer between phases

```
> next phase
```

Autopilot: all phases in one go

- Claude loops phase by phase, gate by gate
- Best for clear plans with solid gates
- Tools: /goal, the /ralph-loop plugin, claude -p in a script
- A notify hook pings me when it is done

```
> Do all the phases in the plan. Stop only if
  a gate fails twice or you need a decision.
```

Start careful. Move to autopilot once the plan and the gates earn your trust.

My tricks and good practices for the loop

1

Numbered plans

001, 002, 003: never rewrite an old plan for new work

2

progress.md every phase

Status, evidence, decisions, known issues: always current

3

Ask only when blocked

Claude picks sensible defaults and says which; I decide the big calls

4

Small sample first

Test on a handful of items before any big or paid run

5

/clear between phases

progress.md holds the memory, so the chat can start fresh

6

Notify me when done

A Stop hook sends a push: leave long runs and come back

7

Skills hold the recipe

initialize-project builds the docs, plans/ and progress.md for me

8

No commit unless asked

I review the diff or file first, then say commit

4

Amplify

Context, flags, MCP, review and parallel work

In this section

- Context management
- Terminal flags
- MCP
- Review commands
- Remote and learning
- Habits

/clear often: keep the context lean

/context

See what fills the context window

/clear

Fresh start between unrelated tasks

/rewind

Roll the conversation back to an earlier point

/memory

View and edit what Claude remembers

/btw

Side question that doesn't clutter the chat

/rename

Name a session so you can resume it later

/goal

Set a goal Claude keeps working toward

Works in the extension chat and in the CLI

Terminal flags: resume, fork, script, and run in parallel

`claude -c`

Continue the most recent conversation

`claude --resume`

Pick an earlier session to resume

`--fork-session`

Branch off a session, keep the original

`claude -p "..."`

One prompt, plain output, for scripts

`claude --agent coordinator`

Run the whole session as that agent

`claude --worktree add-billing`

Isolated git worktree for parallel work

MCP: plug outside tools into Claude, code or not

For code

- GitHub: issues and PRs
- Docs and databases
- Browser testing

Beyond code

- Google Drive: read and write files
- Gmail and Calendar
- Slack, Notion, spreadsheets

What it is

- A standard way to connect Claude to outside tools and data
- Add it once, use it in any chat
- Only add servers you trust

Just ask Claude to install it: "Install the Gmail MCP for me, then summarize my unread emails."

```
claude mcp list                                  # configured servers (in chat: /mcp)
claude mcp add --transport http <name> <url>     # add one; --scope user = all projects
```

Check and ship with the built-in commands

/run

Launch the app and see the change working

/verify

Check that the work does what was asked: code, docs or data

/code-review

Review the diff for bugs; "ultra" runs a deep cloud review

/security-review

Security check of the pending changes

/simplify

Clean up changed code for reuse and clarity

/install-github-app

Connect Claude to your GitHub repo and PRs

Take it with you, and keep learning inside the tool

Anywhere

/remote-control

Continue a running session from another device. Start a long task, then check on it from your phone.

Learn

/powerup

Interactive in-product lessons for features you haven't tried yet. A good way to find new tricks.

Eight habits to take home

1

Open a folder

File > Open Folder..., then start Claude

2

CLAUDE.md

Ask Claude to write it

3

Ralph loop

Docs, plan, progress.md, iterate

4

Exit gates

Verify, self-correct, then "next phase"

5

Keep context lean

/clear, /context, /rewind

6

Config via Claude

Skills, agents, hooks

7

Review

/code-review, /security-review

8

Extend + parallelize

MCP, --worktree, agents